```
from google.colab import files
import pandas as pd

uploaded = files.upload()
```

Choose files medical_dataset.csv
medical_dataset.csv(text/csv) - 2118 bytes, last modified: 01/08/2026 - 100% done
Saving medical_dataset.csv to medical_dataset.csv

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df = pd.read_csv("medical_dataset.csv")
```

```
df
```

	Age	BMI	BloodPressure	Disease
0	58	39.3	104	1
1	36	22.7	169	0
2	69	39.6	159	1
3	75	33.1	174	1
4	42	23.7	159	0
...	...	...	...	...
145	57	38.1	108	1
146	58	38.8	148	1
147	53	21.6	107	0
148	77	33.7	136	1
149	39	34.5	120	0

150 rows × 4 columns

Next steps: Generate code with df New interactive sheet

```
df.head()
```

	Age	BMI	BloodPressure	Disease
0	58	39.3	104	1
1	36	22.7	169	0
2	69	39.6	159	1
3	75	33.1	174	1
4	42	23.7	159	0

Next steps: Generate code with df New interactive sheet

```
df.tail()
```

	Age	BMI	BloodPressure	Disease
145	57	38.1	108	1
146	58	38.8	148	1
147	53	21.6	107	0
148	77	33.7	136	1
149	39	34.5	120	0

```
df.shape
```

```
(150, 4)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Age            150 non-null    int64
 1   BMI            150 non-null    float64
 2   BloodPressure  150 non-null    int64
 3   Disease        150 non-null    int64
dtypes: float64(1), int64(3)
memory usage: 4.8 KB
```

```
df.isnull().sum()
```

	0
Age	0
BMI	0
BloodPressure	0
Disease	0

dtype: int64

```
df.describe()
```

	Age	BMI	BloodPressure	Disease
count	150.000000	150.000000	150.000000	150.000000
mean	50.660000	29.294667	136.653333	0.493333
std	16.327944	6.816106	25.705087	0.501630
min	20.000000	18.000000	90.000000	0.000000
25%	37.250000	23.625000	113.750000	0.000000
50%	52.500000	29.600000	136.000000	0.000000
75%	62.750000	35.400000	160.000000	1.000000
max	79.000000	39.700000	178.000000	1.000000

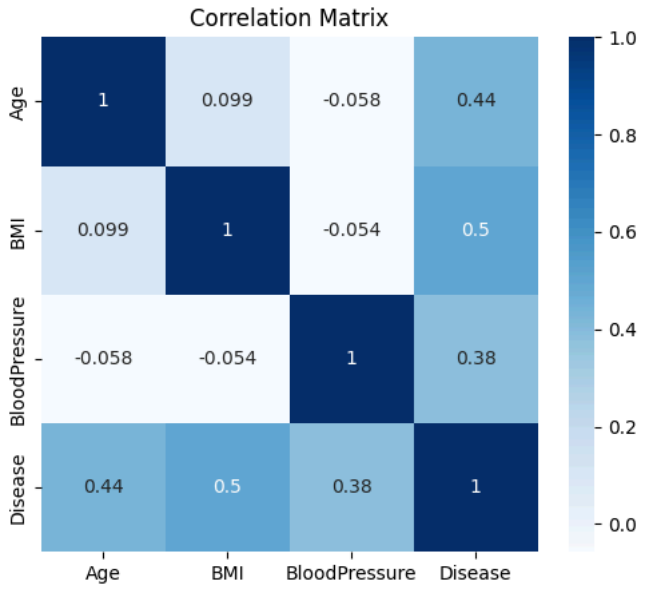
```
df.duplicated().sum()
```

```
np.int64(0)
```

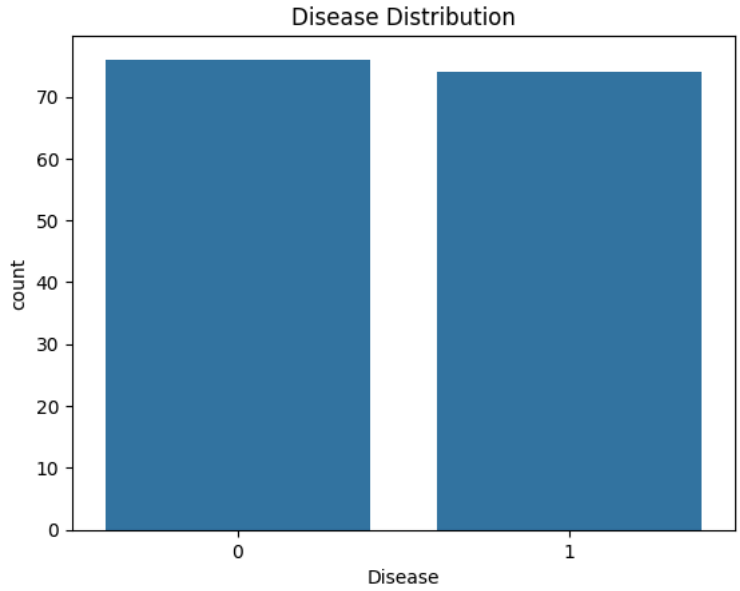
```
df.corr()
```

	Age	BMI	BloodPressure	Disease
Age	1.000000	0.099310	-0.057513	0.435235
BMI	0.099310	1.000000	-0.053841	0.496794
BloodPressure	-0.057513	-0.053841	1.000000	0.383419
Disease	0.435235	0.496794	0.383419	1.000000

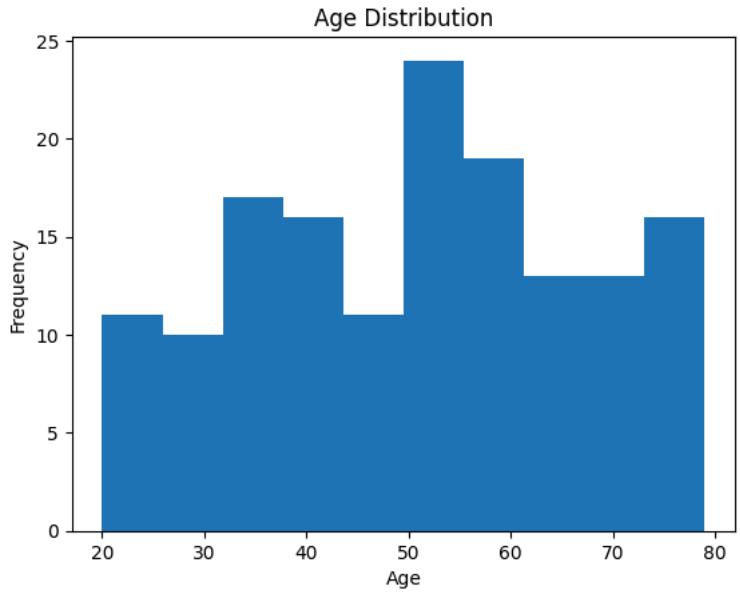
```
plt.figure(figsize=(6,5))
sns.heatmap(df.corr(), annot=True, cmap="Blues")
plt.title("Correlation Matrix")
plt.show()
```



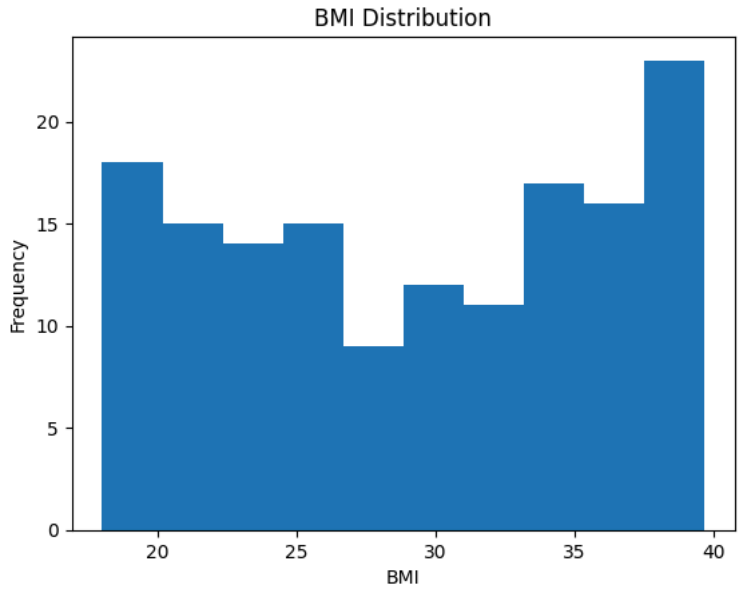

```
sns.countplot(x="Disease", data=df)
plt.title("Disease Distribution")
plt.show()
```



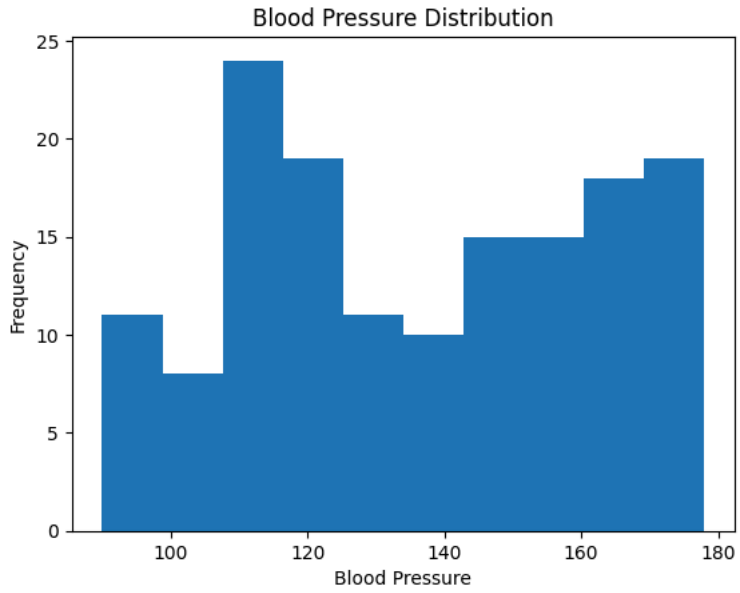

```
plt.hist(df["Age"], bins=10)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



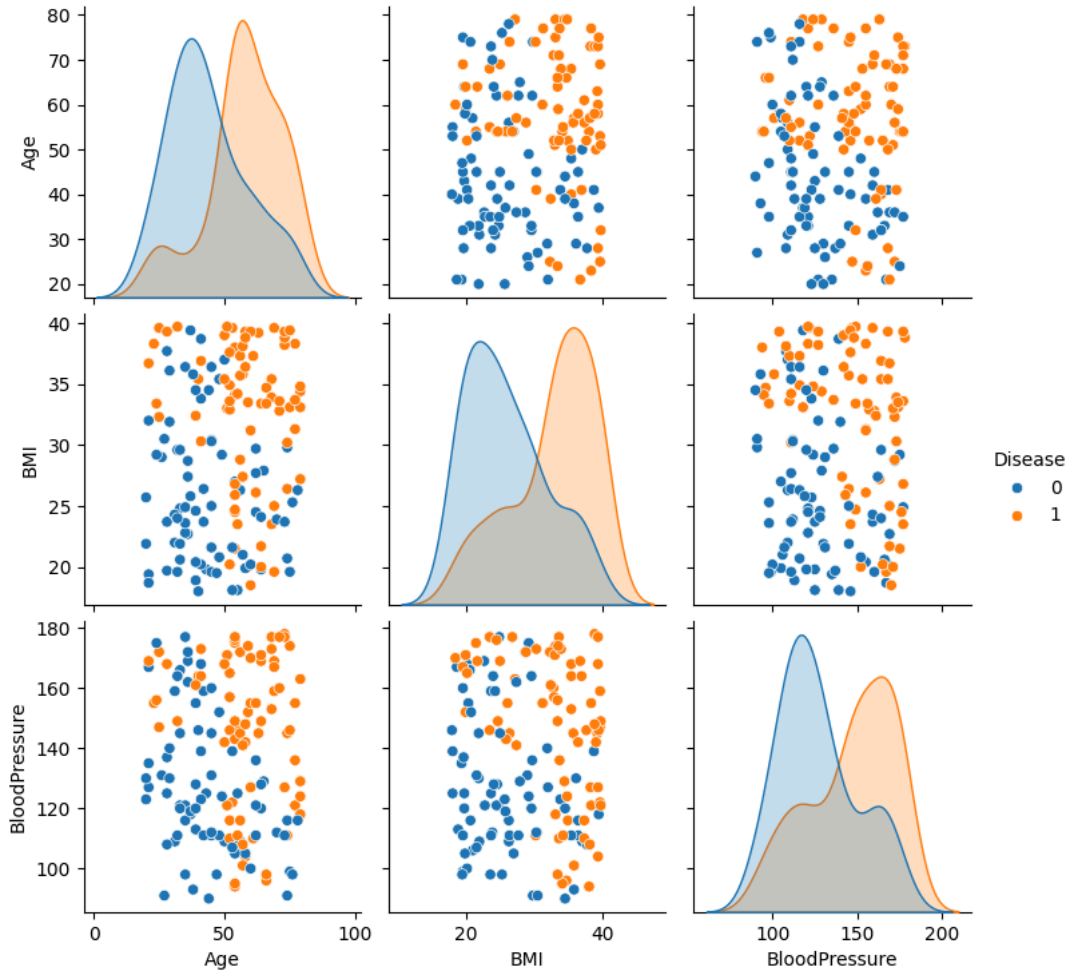

```
plt.hist(df["BMI"], bins=10)
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Frequency")
plt.show()
```




```
plt.hist(df["BloodPressure"], bins=10)
plt.title("Blood Pressure Distribution")
plt.xlabel("Blood Pressure")
plt.ylabel("Frequency")
plt.show()
```




```
sns.pairplot(df, hue="Disease")
plt.show()
```




```
X = df[['Age', 'BMI', 'BloodPressure']]
```

```
y = df['Disease']
```

```
X.head()
```

	Age	BMI	BloodPressure
0	58	39.3	104
1	36	22.7	169
2	69	39.6	159
3	75	33.1	174
4	42	23.7	159

Next steps: Generate code with X | New interactive sheet

```
y.head()
```

	Disease
0	1
1	0
2	1
3	1
4	0

dtype: int64

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(120, 3)
(30, 3)
(120,)
(30,)
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

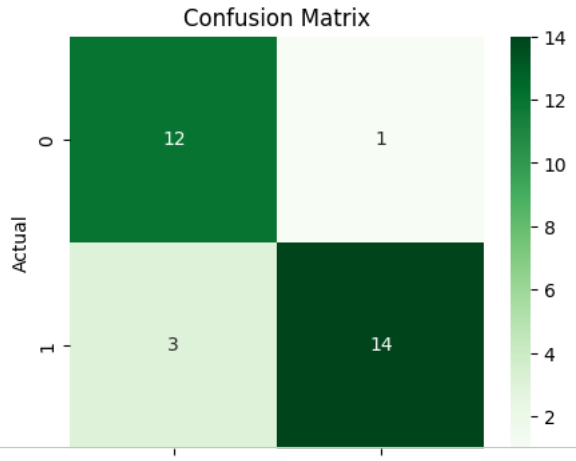
```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8666666666666667
```

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[12  1]
 [ 3 14]]
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.80      0.92      0.86        13
           1       0.93      0.82      0.88        17

    accuracy                           0.87        30
   macro avg       0.87      0.87      0.87        30
weighted avg       0.88      0.87      0.87        30
```

```
print("Coefficients:", model.coef_)
```

```
Coefficients: [[0.22197215 0.56730698 0.13115291]]
```

```
print("Intercept:", model.intercept_)
```

```
Intercept: [-45.92127564]
```

```
patient = pd.DataFrame({
    'Age': [55],
    'BMI': [32],
    'BloodPressure': [145]
})

prediction = model.predict(patient)

print("Prediction:", prediction)
```

```
Prediction: [1]
```

```
if prediction[0] == 1:
    print("Person is susceptible to disease.")
else:
    print("Person is not susceptible to disease.")
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.